

Analysis: Alphabetomials

1. Product of two sets

The problem was created during a short walk in Manhattan in a summer evening. At first there came the picture of an N by M matrix. Let A be a set of N words $\{A_i \mid 1 \leq i \leq N\}$, and B be a set of M words $\{B_j \mid 1 \leq j \leq M\}$. For any word X , let us denote $X(c)$ the number of occurrences of character c in X . In the (i,j) -entry of the matrix, we write, say, the product $A_i('a')B_j('a')B_j('b')$. What is the sum of all the entries in the matrix?

One can compute the NM terms one by one. But it is easy to see, as you can sum them row by row, as well as column by column, it is nothing but the full expansion of the following expression

$$(1) \quad \sum_{X \in A, Y \in B} X('a')Y('a')Y('b') = \sum_{X \in A} X('a') \sum_{Y \in B} Y('a')Y('b').$$

i.e., you can compute the sum on A and B separately, then compute their product.

What if, as in our problem, instead of writing $A_i('a')B_j('a')B_j('b')$ for the (i,j) -entry, we write $Z('a')^2Z('b')$, where Z is the concatenation of A_i and B_j ? i.e., we do not care which 'a's are from the first set, and which are from the second. In this case we do not have something as simple as (1). But, we can reduce it if we *do care* which of the 'a's are from the first set. We use a somehow simplified notation (which is closer to our problem statement) -- the symbol a now also represent the number of 'a's in a string. For a fixed string Z that is a concatenation of words from A and B , let a_1 denote the number of 'a's contributed by the first set, and a_2 the number of 'a's contributed by the second set. For each entry in the matrix, we write a^2b , this is equal to $(a_1+a_2)^2(b_1+b_2)$. Expand it, we have

$$\begin{aligned} (2) \quad \sum_{X \in A, Y \in B} a^2b &= \sum_{X \in A, Y \in B} (a_1+a_2)^2(b_1+b_2) \\ &= \sum_{X \in A, Y \in B} (a_1^2b_1 + a_1^2b_2 + 2a_1a_2b_1 + 2a_1a_2b_2 + a_2^2b_1 + a_2^2b_2) \\ &= \sum_{A,B} (a_1^2b_1) + \sum_{A,B} (a_1^2b_2) + 2\sum_{A,B} (a_1a_2b_1) + \\ &\quad 2\sum_{A,B} (a_1a_2b_2) + \sum_{A,B} (a_2^2b_1) + \sum_{A,B} (a_2^2b_2) \\ &= \sum_A a^2b \sum_B 1 + \sum_A a^2 \sum_B b + 2\sum_A ab \sum_B a + \\ &\quad 2\sum_A a \sum_B ab + \sum_A b \sum_B a^2 + \sum_A 1 \sum_B a^2b \end{aligned}$$

In the last step, for each summand, since the characters from the first set and those from the second set are nicely separated, we can apply the same reason for (1).

2. Product of ten sets

Our problem asks to compute the sum of expressions such like a^2b over the product of a dictionary set by itself up to 10 times. (Or more generally, we can think of the product of 10 sets.) The size of the product set is the huge number n^{10} . But if we use the same trick as in the previous section, write a^2b as

$$(a_1 + a_2 + \dots + a_{10})^2 (b_1 + b_2 + \dots + b_{10})$$

and expand it. Then we reduce the problem to at most K^D easier problems, where D is the maximal degree ($K \leq 10$ and $D \leq 4$ in our problem). Each of the easier problems can be solved similar as in (1).

This gives one solution to our problem. Focus on one monomial a time. We are basically choosing the origin (one of the ten sets) for each letter in the monomial, so each set gets a sub-monomial, which can be expressed as a subset of the indices of the letters in the original monomial.

We pre-compute $\sum_{X \in A} q(X)$ for each sub-monomial q , there are at most 16 of these. Then for each of the K^D "easier problems", the computation involves only multiplying K of the pre-computed numbers.

3. Speedups

The solution above can be easily sped up to solve for much bigger K and slightly bigger D , if one compute the summations incrementally, do the product of two sets in each step (you call it dynamic programming or not).

First, we compute the sum for $A \times A$, not only for the monomial, but for all its sub-monomials.

Then we compute the sums over A^3 . Now we can view A^3 as $A^2 \times A$. Each sub-monomial can be expanded to at most 2^D terms, and every one of them submit to the simple trick in (1).

Because we already have the summation of A^2 on any sub-monomial, each of the 2^D problems can be solved in constant time. The running time of this solution is mostly dominated by $O(3^D K)$ for each monomial form the input.

Observe the last line in (2), notice that we only have one dictionary set in our problems, so we can ignore the subscripts -- A , B , etc are the same set. There are other speedups available along this line by exploring the symmetry in indices. We omit the details.

4. The theory behind it

In case you are familiar with discrete probability, you may well know that probability theory provides some powerful abstract machinery for counting. What we went through so far are just some easy exercises in probability. And solving this problem with the abstract theory in mind certainly speeds up your thinking and boosts your confidence in the solution.

Especially, we went through the important fact that the expectation of the product of K random variables equals to the product of the expectation of each of the variables, given that the K random variables are mutually independent. The interested readers may carry the formal correspondences out.

Analysis: EZ-Sokoban

This is a state space search problem: given a set of states (positions on the board), an initial state and a final state, and rules for state transformations, find a sequence of moves that transforms the initial state to the final state. In our case, the problem asks for the length of the shortest such sequence of moves.

Conceptually, we can represent such a state space as a graph. The nodes of the graph are the possible positions, and the edges are the allowed moves. The problem then becomes: find the shortest path in the graph. The standard algorithm to solve this graph problem is [breadth-first search](#).

Let's estimate the number of nodes in the graph. Assuming the maximum number of boxes (5), first estimate the number of positions where all the boxes are connected. 5 connected boxes form a [pentomino](#). There are 63 different pentominoes, counting all rotations and reflections. Each of these can be positioned at no more than 12×12 different positions. Hence we get an upper bound of $63 \times 12 \times 12 = 9072$ connected positions. It is a little more difficult to estimate the "dangerous" positions accurately, but we can see that from each connected position there are not too many moves, so we can guess that the total is not going to be too large for our computer to handle.

One approach would be to first generate the graph with all the edges explicitly, and then run the breadth-first-search on it. Another is to not store the graph at all, but compute the possible moves (edges) from a given position as we go, and only store the set of visited positions in a data structure.

Some details to work out are:

- How to represent positions. A simple list of box coordinates can work. One can also use a whole-board bit-mask.
- How to look up positions. We need this to see if a position has already been visited, or to avoid constructing the same node in the graph multiple times. We can use some kind of a dictionary data structure - a hash table or a binary search tree.
- How to generate moves. Just try moving all the boxes in every direction, if the space in front and in the back of the box is empty. We also have to make sure that we don't move from a dangerous position to another dangerous position.
- How to check whether a position is dangerous. To do this, we need to check if our 1 to 5 boxes are all connected. This can be represented as another, small graph problem ([graph connectivity](#)). We run another breadth-first-search on the little graph, where the nodes are the boxes and the edges indicate whether two boxes touch. Another approach would be to pre-generate all [polyominoes](#) of sizes up to 5, store them in a hash table, and then look up the shape appearing in a given position.

Analysis: Football Team

0. Drawing the graph

It is clear from the statement that this is a graph coloring problem -- on 1000 vertices. But certainly this is not the end of the story. There must be something special that makes this famous NP-complete problem solvable in this contest.

Represent each player with a vertex, and draw an edge between two players if they cannot use the same color. Obviously we can just draw the graph with each player in his position (x, y) . So there are at most 30 rows of players. Can this help? Observing this fact leads to solutions that are good enough for the small dataset, but not for the large tests.

On each row, the edges form a horizontal line segment from the leftmost point to the rightmost. All the other edges are between two adjacent rows. And more importantly, it is not hard to observe and prove the following:

1. If there are at least two edges between two adjacent rows, then all these edges together with the horizontal row edges form triangles.
2. If you draw all the edges using straight line segments, none of the edges cross each other in the middle.

So, surprising or not, the graph is planar! There are special cases where we can handle easily:

- One color is enough if and only if there are no edges.
- In a general graph, it is well known that the graph is 2-colorable if and only if there are no odd cycles. In our problem, it is even simpler than that. One can prove that whenever there is a cycle, there must also exist a triangle. So, the graph is 2-colorable if and only if it is a tree, as well as if and only if there is no triangles.

Otherwise, the answer is at least 3. But you may also know very well about one of the famous theorems of the past century, the Four Color Theorem: 4 colors is enough to color any planar graph.

So this is a good news: besides the simple special cases, all we need to decide is whether the answer is 3 or 4. Unfortunately it is also known that even this problem is NP-complete in general.

As we mentioned earlier, we are dealing with a special case of even planar graphs. It is mostly triangulated, and nicely positioned in rows with all edges only from adjacent rows. It turns out that there are many solutions to this problem. Let us describe a few in the rest of this writing. In all the solutions, we try to color the graph with just 3 colors A, B, and C, and if this fails, the answer is 4. Also, to simplify our writing, we assume every point has degree at least 3 -- otherwise we can simply remove it without changing the 3-colorability.

1. The colors are pretty much forced

Pick any triangle. Its vertices must be colored with different colors. We color them with A, B, and C. Now, there might be another triangle *next to it*. i.e., sharing an edge e with it. In that triangle, two vertices of e are already colored; if the third one is not, its color is decided now!

From another point of view, this is simply the following: Whenever there is a vertex with two *adjacent* neighbors colored, we color it with the third color.

Because every color is forced, the answer is 4 as soon as we see a conflict.

We can do this forced coloring almost all the way. It is easy to see we only need to pause a little when there is a cut point. But this is not hard either: the point separates the graph into an upper part and a lower part, as long as we can color both parts with 3 colors, we can paste them together.

2. The dual graph

There is a deeper concept revealed in the solution above. We were actually considering the dual graph of a planar graph, which itself is also planar. Each face (in our problem, each triangle) in the original graph becomes a point, and two points are connected if the two faces share an edge.

If you are familiar with the algorithm that 2-colors a graph and detects odd cycles, you may find it very similar to our solution above. We are going to argue that our original graph is 3-colorable if and only if the dual graph is 2-colorable (with one color for all the clockwise triangles).

Observe that if the original graph is 3-colorable, and one triangle gets its three vertices colored A, B, C in clockwise order, then any triangle adjacent to it *must* have its vertices colored A, B, C in counter-clockwise order. Thus, the original graph being 3-colorable implies its dual is 2-colorable.

The proof of the reversed direction is also not hard to envisage, but it involves more details and requires some additional conditions on our graph. One easy way to prove it is with induction on rows.

3. The local property

The solution above does not allow a much simpler implementation. But it leads to an even nicer observation. We call a vertex in the original graph an *inner vertex* if it is surrounded by triangles in all the directions. The following fact gives the simplest solution to our problem:

The graph is 3-colorable if and only if there is no inner vertex with odd degree.

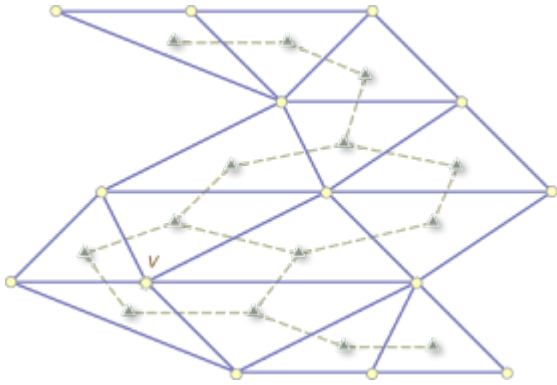
The rest of this section is devoted to the proof of this fact. If you feel that some of the claims about the planar graph and its dual are not obvious, try to draw some and convince yourself.

Any inner vertex v with degree d is surrounded by d triangles sharing the same vertex. In the dual, they give a cycle of length d that enclose v in the middle.

One direction of the proof is easier. If v has odd degree, then we see an odd cycle in the dual, therefore the original graph is not 3-colorable.

Now assume there is any odd cycle, let us pick C -- the one with the *smallest enclosing area*. We claim that C must enclose only one inner vertex, and therefore we find an inner point with odd degree. This can be proved in several ways with some details. Basically, if there are two inner vertices enclosed, one can find a path that separates them and hit C in two places. This divides the enclosing area of C into two smaller areas, and it is not hard to see one of the areas must be enclosed by an odd cycle. This contradicts the assumption that C is the odd cycle with smallest area, and completes the sketched proof of our short solution.

In the picture below, there is a big cycle with length 9, and v is an inner vertex surrounded by a cycle with length 5.



4. Other solutions

We mention two brute-force solutions that are good enough for the small dataset. In order to use them for the large dataset, one needs to optimize a great deal, and possibly just discover one of keys for the polynomial solutions as mentioned above.

Again, we look at the original graph. We assume the graph is connected, and every vertex has degree at least 3.

Notice that there are at most 30 rows. One can fix the color of the first vertex in each row. There are 2^{rows} ways of doing this (as opposed to 3^{rows}). Once this is fixed, all the rest of the coloring is determined, for a reason similar to the one explained in Section 1.

Another solution is dynamic programming: do a sweep from the right to the left, and remember the color of the first vertex to the right on each row. Notice that you cannot do this from left to right, since knowing the color of only one vertex on a row is not enough in that direction.

More information

[Graph coloring](#) - [Four color theorem](#) - [Dual graph](#)

The solution with the local property resembles very much a theorem of Heawood from 1898: that a maximal planar graph is 3-colorable if and only if every degree is even. A more general theorem, which implies our solution, can be found in the paper *A new 3-color criterion for planar graphs* by Krzysztof Diks, Lukasz Kowalik, and Maciej Kurowski.

Analysis: Interesting Ranges

1. How big can the answer be?

First, let's get a crude upper bound on the answer. More specifically, if $L=1$ and $R=N$, there are $N(N+1)/2$ subsegments of $[L,R]$. Not all of them will contain an even number of palindromes, but a significant part (roughly half) will.

So even for the small input we might get more than 10^{25} segments in the answer; it's obvious we can't enumerate them one by one. We have to figure out a way to process them in bulks.

2. First optimization

The first idea required to solve the small dataset helps us reduce the number of segments to consider from $O(N^2)$ to $O(N)$. To achieve that, we rely on the following observation: $[L,R]$ can be represented as $[0,R]$ minus $[0,L-1]$; thus it contains an even number of palindromes if and only if $[0,L-1]$ and $[0,R]$ both contain even or both contain odd number of palindromes.

But how exactly does that help us? Suppose we know which of $[0,X]$ segments contain even number of palindromes (we'll call them just "even 0-segments" further on) and which contain odd number of palindromes ("odd 0-segments"). We know that each interesting segment corresponds to exactly one pair of even 0-segments or to exactly one pair of odd 0-segments. But this is true the other way around as well: each pair of distinct even 0-segments corresponds to exactly one interesting segment, and so does each pair of distinct odd 0-segments! (when X is between $L-1$ and R , inclusive).

That means that if there are A even 0-segments and B odd 0-segments, the answer is $A(A-1)/2 + B(B-1)/2$.

2. Second optimization

But we can't even afford $O(N)$ running time since N is 10^{13} even in the small dataset! So we have to do another optimization.

Let's write out an infinite string of zeroes and ones, with X -th symbol (0-based) being equal to 0 if $[0,X]$ contains an even number of palindromes, and equal to 1 if $[0,X]$ contains an odd number of palindromes. What we need to find in this problem is how many zeroes (number A above) and ones (number B above) are there in the substring starting with $(L-1)$ -th character and ending with R -th character of this string.

This string looks like this: 101010100111111111110000000000111111111100...

Now we can spot the second optimization: zeroes and ones tend to go in big blocks in this string. More specifically, one changes to zero or vice versa only when we pass a palindrome. And there are only $O(\sqrt{N})$ palindromes up to N - so the number of groups of consecutive zeros or ones in the first N characters is $O(\sqrt{N})$.

All groups except maybe two boundary ones fit into $[L-1,R]$ segment entirely. So we just need to sum them all and handle the boundary ones carefully, and we get an $O(\sqrt{N})$ algorithm that is sufficient to solve the small dataset.

We can also reduce the number of boundary groups to consider from two to one relying on the fact that the number of zeroes/ones in $[L-1, R]$ is the number of zeroes/ones in $[0, R]$ minus the number of zeroes/ones in $[0, L-2]$.

3. Third optimization

So what about the large one? $\text{Sqrt}(10^{100})=10^{50}$, so we're definitely not there yet.

The final optimization idea still relies on the above infinite string of zeroes and ones. You might have noticed already that many blocks of ones and zeroes have the same length. For example, block of ones from 11 to 21 has length 11, and so is the block of zeroes from 22 to 32, ones from 33 to 43, zeroes from 44 to 54, and so on.

This is because of the fact that a palindrome number is uniquely determined by its first half. For example, consider 6-digit palindrome number 127721. What is the next 6-digit palindrome number? 128821. It's followed by 129921, 130031, and so on. As you can see, in most cases the difference between two consecutive 6-digit palindrome numbers is 1100 (change of +1 in two middle digits). And the difference between two consecutive palindrome numbers is exactly the length of the block of ones/zeroes!

But there are also some blocks which have length different from usual. For example, the block of zeroes from 88 to 98 still has 11 numbers, but the block of ones from 99 to 100 has just two numbers; then follow several blocks of 10 (zeroes from 101 to 110; ones from 111 to 120; ...; zeroes from 181 to 190), then we have a block of 11 ones from 191 to 201.

The above explanation about consecutive palindromes allows us to understand why there are such unusual-length blocks. It happens in two cases: when the amount of digits in the palindrome changes, and when the middle digit of the starting palindrome of the block is 9. The first several unusual-length blocks are: zeroes from 9 to 10, ones from 99 to 100, ones from 191 to 201, ones from 292 to 302, ..., ones from 898 to 908, ones from 999 to 1000, ones from 1991 to 2001, ones from 2992 to 3002, and so on.

And here comes the final step. All such unusual-length blocks except the one from 9 to 10 consist of ones! Or, in other words, all blocks of zeroes have the same length within palindromes of the same amount of digits, except the block from 9 to 10.

Why? Because there's ten blocks between two consecutive palindromes with a 9 in the middle (with the only exception being 9 and 99), and ten is an even number. That's why every time we have a 9 in the middle, we start a block of ones.

And that allows us to calculate in one step the total amount of zeroes in the part of our infinite string that corresponds to one amount of digits in the palindrome. That means we can calculate the overall amount of zeroes in any segment in $O(\text{number of digits in the maximal palindrome})$ steps. Of course, we have to be careful near L and R .

And what about the amount of ones? It's equal to the total length minus the amount of zeroes :)

4. Modulo calculations

With the numbers in the input being quite big, the above calculations have to be performed carefully.

Some programming languages allow easy calculations with arbitrarily long numbers. But what if we use a language without this feature? Luckily, we're asked to find the answer modulo a (relatively) small number M .

That allows us to perform most calculations modulo M , and thus use only small numbers in those calculations. The calculations we need to find the answer are: addition, multiplication, subtraction and division by 2. The first three are performed in a standard way (first calculate the answer in integers, then take it modulo M). The fourth one is made possible by the fact that the modulo used in the problem is odd. When we divide a number X by 2 modulo M , we get just $X/2$ if X is even, and $(X+M)/2$ when x is odd. One can easily check that multiplying by 2 gets X back in both those cases.